Moragora API 명세서

Express + Supabase + Multi-AI Judgment System — 토론 판결 플랫폼 백엔드 API (68 endpoints)

목차 (17개 섹션, 68개 엔드포인트)

1. 시스템 개요
2. 인증 방식
3. 데이터베이스 스키마
4. 인증 API (/api/auth) [3]
5. 토론 API (/api/debate) [12]
6. 주장 API (/api/argument) [3]
7. 채팅 API (/api/chat) [7]
8. 판결 API (/api/judgment) [8]
9. 투표 API (/api/vote) [6]
10. 댓글 API (/api/comment) [3]
11. 프로필 API (/api/profile) [12]
12. 알림 API (/api/notification) [6]
13. 콘텐츠 필터 API (/api/content) [2]
14. 피드백 API (/api/feedback) [2]
15. 분석 API (/api/analytics) [2]
16. 관리자 API (/api/admin) [4]
17. AI API (/api/ai) [3]
18. OG 메타 API (/api/og) [1]
19. 크론 API (/api/cron) [3]
20. AI 판결 시스템

21. 에러 코드 정리

1. 시스템 개요

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Client (React + Vite)                         │
│                    Vercel: team-moragora-client                      │
└──────────────────────────────┬───────────────────────────────────────┘
                               │  HTTPS (axios + JWT)
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      Express Server (Render)                         │
│  Port: 5000  |  CORS: CLIENT_URL  |  Base: /api                      │
├──────────────────────────────────────────────────────────────────────┤
│  Middleware: requireAuth │ optionalAuth │ contentFilter │ errorHandler│
├────┬──────┬────┬──────┬──────┬────┬────┬──────┬──────────────────────┤
│/auth│/debate│/arg.│/chat │/judg. │/vote │/comm.│/prof. │/notif.        │
├────┴──────┴────┴──────┴──────┴────┴────┴──────┴──────────────────────┤
│/content│/feedback│/analytics│/admin│/ai│/og│/cron                     │
└────┬────┴────┬────┴────┬────┴────┬──┴──┴──┴──┬──────────────────────┘
     │         │         │         │            │
     ▼         ▼         ▼         ▼            ▼
┌──────────────────────────────────────────────────────────────────────┐
│                     Supabase (PostgreSQL + Auth)                     │
│  Tables: profiles, debates, arguments, verdicts, ai_judgments,       │
│          votes, comments, notifications, content_filter_logs,        │
│          xp_logs, feedbacks, analytics_events, chat_messages         │
└──────────────────────────────────────────────────────────────────────┘
                               │
                 ┌─────────────┼─────────────┐
                 ▼             ▼             ▼
            ┌────────┐   ┌──────────┐   ┌────────┐
            │ GPT-4o │   │Gemini2.5 │   │ Claude │   ← 3개 AI 병렬 판결
            │Judge G │   │ Judge M  │   │Judge C │
            └────────┘   └──────────┘   └────────┘
                 │             │             │
                 └─────────────┼─────────────┘
                               ▼
                        ┌──────────────┐
                        │  Grok 3 Mini│  ← 실패 시 대체 판사
                        │  Judge X    │
                        └──────────────┘
```

항목	내용
서버	Express.js (Node.js) — Render 배포
데이터베이스	Supabase (PostgreSQL + Auth + RLS)
프론트엔드	React + Vite — Vercel 배포
인증	Supabase Auth (JWT Bearer Token)

AI 판결	GPT-4o, Gemini 2.5 Flash, Claude Sonnet 4 (병렬) + Grok 3 Mini (폴백)
콘텐츠 필터	3단계 (사전 필터 → AI 유해성 → AI 주제 적합성)
입력 전처리	프롬프트 인젝션 방어 + 텍스트 정규화 + 유사도 검증
총 엔드포인트	68개 (17개 라우트 그룹)

2. 인증 방식

Supabase Auth 기반 JWT 토큰 인증을 사용합니다.

Request Header

```
Authorization: Bearer <supabase_access_token>
```

인증 미들웨어

미들웨어	동작	실패 시
`requireAuth`	JWT 검증 필수, req.user 설정	`401` Unauthorized
`optionalAuth`	JWT 있으면 검증, 없으면 통과 (req.user = null)	통과 (인증 없이 접근 가능)
`requireAdmin`	requireAuth 후 관리자 권한 확인	`403` Forbidden
`verifyCronSecret`	Cron 비밀키 헤더 검증	`401` Unauthorized
`contentFilter`	콘텐츠 3단계 필터링	`400` Bad Request
`commentFilter`	댓글 전용 필터링	`400` Bad Request

> **참고:** 공개 API(토론 목록, 판결 피드, 랭킹 등)는 인증 없이 접근 가능합니다. 각 엔드포인트의 `AUTH` / `PUBLIC` 뱃지를 확인하세요.

3. 데이터베이스 스키마 ▼

PK **profiles**

컬럼	타입	제약조건	설명
id	UUID	PK, FK auth.users CASCADE	사용자 ID
nickname	TEXT	NOT NULL, DEFAULT ''	닉네임
gender	TEXT	CHECK IN ('male','female','other')	성별
age	INT	CHECK 1~150	나이
avatar_url	TEXT	NULL	프로필 이미지 URL
wins	INT	NOT NULL, DEFAULT 0	승리 수
losses	INT	NOT NULL, DEFAULT 0	패배 수
draws	INT	NOT NULL, DEFAULT 0	무승부 수
total_score	INT	NOT NULL, DEFAULT 0	총 점수 (XP 누적)
tier	TEXT	NOT NULL, DEFAULT '시민', CHECK IN ('시민','배심원','변호사','판사','대법관')	등급
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	가입일

PK debates

컬럼	타입	제약조건
id	UUID	PK, DEFAULT gen_random_uuid()
creator_id	UUID	FK profiles CASCADE, NOT NULL
opponent_id	UUID	FK profiles CASCADE, NULL
topic	TEXT	NOT NULL
description	TEXT	NULL
pro_side	TEXT	NULL
con_side	TEXT	NULL
vote_duration	INT	NULL
category	TEXT	NOT NULL, DEFAULT 'daily'
purpose	TEXT	NOT NULL, DEFAULT 'compete'
lens	TEXT	NOT NULL, DEFAULT 'general'
mode	TEXT	NOT NULL, DEFAULT 'duo', CHECK IN ('duo','solo','daily')
invite_code	TEXT	UNIQUE, NOT NULL
status	TEXT	NOT NULL, DEFAULT 'waiting', CHECK IN ('waiting','both_joined','arguing','judging','voting','completed')
vote_deadline	TIMESTAMPTZ	NULL
view_count	INT	NOT NULL, DEFAULT 0
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()

PK **arguments**

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	주장 ID
debate_id	UUID	FK debates CASCADE, NOT NULL	토론 ID
user_id	UUID	FK profiles CASCADE, NOT NULL	작성자
side	TEXT	NOT NULL, CHECK IN ('A','B')	진영
round	INT	NOT NULL, DEFAULT 1, CHECK IN (1,2)	라운드 (R1: 2000자, R2: 300자)
content	TEXT	NOT NULL, CHECK 1~2000자	주장 내용 (전처리 완료)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	작성일

UNIQUE (debate_id, side, round)

PK **verdicts**

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	판결 ID
debate_id	UUID	FK debates CASCADE, UNIQUE NOT NULL	토론 ID (1:1)
winner_side	TEXT	NOT NULL, CHECK IN ('A','B','draw')	최종 승자
summary	TEXT		판결 요약
ai_score_a	INT	NOT NULL, DEFAULT 0	AI 종합 점수 A
ai_score_b	INT	NOT NULL, DEFAULT 0	AI 종합 점수 B
citizen_score_a	INT	DEFAULT 0	시민 투표 점수 A
citizen_score_b	INT	DEFAULT 0	시민 투표 점수 B
final_score_a	NUMERIC(5,2)	NOT NULL, DEFAULT 0	최종 점수 A (AI 75% + 시민 25%)
final_score_b	NUMERIC(5,2)	NOT NULL, DEFAULT 0	최종 점수 B
citizen_vote_count	INT	NOT NULL, DEFAULT 0	시민 투표 수
is_citizen_applied	BOOLEAN	NOT NULL, DEFAULT false	시민 투표 반영 여부
nickname_a	TEXT	NULL	A측 닉네임 스냅샷
nickname_b	TEXT	NULL	B측 닉네임 스냅샷
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	판결일
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	수정일

PK **ai_judgments**

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	AI 판결 ID
verdict_id	UUID	FK verdicts CASCADE, NOT NULL	판결 ID
ai_model	TEXT	NOT NULL	gpt-4o / gemini-2.5-flash / claude-sonnet / grok-3-mini
winner_side	TEXT	NOT NULL, CHECK IN ('A','B','draw')	개별 AI 판정
verdict_text	TEXT		판결문 (캐릭터별 톤)
score_a	INT	NOT NULL, DEFAULT 0	A측 총점
score_b	INT	NOT NULL, DEFAULT 0	B측 총점
score_detail_a	JSONB		{logic, evidence, persuasion, consistency, expression}
score_detail_b	JSONB		{logic, evidence, persuasion, consistency, expression}
verdict_sections	JSONB	DEFAULT '[]'	판결문 섹션별 분석
confidence	NUMERIC(4,2)	DEFAULT 0.0	판결 확신도 (0.00~1.00)
status	TEXT	NOT NULL, DEFAULT 'success'	판결 상태 (success/error)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	판결일

PK votes

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	투표 ID
debate_id	UUID	FK debates CASCADE, NOT NULL	토론 ID
user_id	UUID	FK profiles CASCADE, NOT NULL	투표자
voted_side	TEXT	NOT NULL, CHECK IN ('A','B')	투표한 진영
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	투표일

UNIQUE (debate_id, user_id)

PK xp_logs

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	로그 ID
user_id	UUID	FK profiles CASCADE, NOT NULL	사용자
xp_amount	INT	NOT NULL	XP 변동량
reason	TEXT	NOT NULL, CHECK IN ('victory','defeat','draw','daily_debate','vote','bonus','penalty')	사유
reference_id	UUID	NULL	관련 토론 ID
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	기록일

PK content_filter_logs

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	로그 ID
user_id	UUID	FK profiles CASCADE	사용자
debate_id	UUID	FK debates	토론 ID
content_type	TEXT	NOT NULL	콘텐츠 유형
filter_stage	INT	NOT NULL, CHECK IN (1,2,3)	필터 단계
result	TEXT	NOT NULL, DEFAULT 'pass'	결과 (pass/block)
reason	TEXT	NULL	차단 사유
blocked_text	TEXT	NULL	차단된 원문
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	기록일

PK comments

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	댓글 ID
debate_id	UUID	FK debates CASCADE, NOT NULL	토론 ID
user_id	UUID	FK profiles CASCADE, NOT NULL	작성자
content	TEXT	NOT NULL	댓글 내용
likes_count	INT	NOT NULL, DEFAULT 0	좋아요 수
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	작성일

PK notifications

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	알림 ID
user_id	UUID	FK profiles CASCADE, NOT NULL	사용자
type	TEXT	NOT NULL	알림 유형
title	TEXT	NOT NULL	알림 제목
message	TEXT	NULL	알림 내용
link	TEXT	NULL	이동 링크
is_read	BOOLEAN	NOT NULL, DEFAULT false	읽음 여부
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	생성일

PK feedbacks

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	피드백 ID
user_id	UUID	FK profiles CASCADE, NOT NULL	사용자
satisfaction	INT	NOT NULL, CHECK 1~5	만족도
ai_accuracy	INT	NOT NULL, CHECK 1~5	AI 정확도
ui_ease	INT	NOT NULL, CHECK 1~5	UI 편의성
fairness	INT	NOT NULL, CHECK 1~5	공정성
recommend	INT	NOT NULL, CHECK 1~5	추천도
best_feature	TEXT	NULL	최고 기능
improvement	TEXT	NULL	개선점
additional	TEXT	NULL	추가 의견
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	작성일

PK chat_messages

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	메시지 ID
debate_id	UUID	FK debates CASCADE, NOT NULL	토론 ID
user_id	UUID	FK auth.users CASCADE, NOT NULL	작성자
nickname	TEXT	NOT NULL	작성자 닉네임
content	TEXT	NOT NULL	메시지 내용 (200자 제한)
side	TEXT	NOT NULL, CHECK IN ('A','B')	진영
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	작성일

PK verdict_ratings

컬럼	타입	제약조건	설명
id	UUID	PK, DEFAULT gen_random_uuid()	평가 ID
verdict_id	UUID	FK verdicts CASCADE, NOT NULL	판결 ID
user_id	UUID	FK profiles CASCADE, NOT NULL	평가자
score	NUMERIC(2,1)	NOT NULL, CHECK 0.5~5.0	별점 (0.5 단위)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	평가일

UNIQUE (verdict_id, user_id)

추가 테이블: debate_likes, comment_likes, page_views, analytics_events, kicked_users 등 총 18개 테이블. 상세 스키마는 테이블명세서를 참조하세요.

4. 인증 API (/api/auth) — 3개

GET

/api/auth/me

AUTH

내 프로필 조회

설명

현재 인증된 사용자의 프로필 정보를 조회합니다.

RESPONSE 200

```
{
  "id": "uuid",
  "nickname": "string",
  "gender": "male | female | other | null",
  "age": 25,
  "avatar_url": "string | null",
  "wins": 0, "losses": 0, "draws": 0,
  "total_score": 0,
  "tier": "시민",
  "created_at": "2026-04-02T00:00:00Z"
}
```

ERROR

401	인증 토큰 없음 또는 유효하지 않음

PATCH

/api/auth/me

AUTH

프로필 수정

REQUEST BODY

```
{
  "nickname": "string (optional)",
  "avatar_url": "string (optional)"
}
```

RESPONSE 200

수정된 프로필 객체 반환 (GET /api/auth/me와 동일 형태)

POST

/api/auth/logout

AUTH

로그아웃

설명

현재 사용자의 모든 세션을 무효화합니다.

RESPONSE 200

```
{
  "message": "로그아웃 되었습니다."
}
```

5. 토론 API (/api/debate) — 12개

GET

/api/debate/

PUBLIC

토론 목록 조회

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "string",
    "category": "daily",
    "status": "completed",
    "invite_code": "Ab3xK9mZ",
    "created_at": "ISO8601",
    "creator": { "nickname": "닉네임" }
  }
]
```

GET

/api/debate/my/active

AUTH

내 활성 토론 조회

설명

현재 사용자가 참여 중인 활성 상태(waiting/arguing/judging/voting) 토론 목록을 조회합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "string",
    "status": "arguing",
    "mode": "duo",
    "invite_code": "Ab3xK9mZ",
    "created_at": "ISO8601"
  }
]
```

GET

/api/debate/chat/rooms

PUBLIC

채팅방 목록 조회

설명

현재 활성화된 채팅 토론 목록을 조회합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "string",
    "status": "arguing",
    "mode": "chat",
    "participant_count": 5,
    "created_at": "ISO8601"
  }
]
```

GET

`/api/debate/invite/:inviteCode`

PUBLIC

초대코드로 토론 조회

PATH PARAMS

이름	타입	설명
`inviteCode`	String (8자)	초대 코드

RESPONSE `200`

토론 상세 객체 반환

ERROR

`404`	초대 코드에 해당하는 토론 없음

GET

/api/debate/:id

PUBLIC

토론 상세 조회

PATH PARAMS

이름	타입	설명
id	UUID	토론 ID

RESPONSE 200

```
{
  "id": "uuid",
  "topic": "string",
  "description": "string | null",
  "category": "일상",
  "purpose": "승부",
  "lens": "일반",
  "mode": "duo",
  "status": "arguing",
  "invite_code": "Ab3xK9mZ",
  "creator": { "id": "uuid", "nickname": "...", "tier": "시민" },
  "opponent": { "id": "uuid", "nickname": "...", "tier": "배심원" }
}
```

ERROR

404	토론을 찾을 수 없음

POST

/api/debate/

AUTH

토론 생성

REQUEST BODY

```
{
  "topic": "짜장면 vs 짬뽕, 어느 쪽이 더 나은 선택인가?",
  "description": "점심 메뉴 고민에서 시작된 논쟁",
  "category": "일상",
  "purpose": "승부",
  "lens": "일반",
  "mode": "duo"
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "creator_id": "uuid",
  "opponent_id": null,
  "topic": "짜장면 vs 짬뽕...",
  "invite_code": "Ab3xK9mZ",
  "status": "waiting",
  "created_at": "2026-04-02T..."
}
```

상태 흐름: waiting → arguing → judging → voting → completed

Solo 모드: `mode="solo"` 일 경우 바로 `status="arguing"` 으로 시작됩니다.

POST

`/api/debate/join/:inviteCode`

AUTH

초대코드로 참여

PATH PARAMS

이름	타입	설명
`inviteCode`	String (8자)	초대 코드

비즈니스 로직

1	invite_code + status=waiting + opponent_id=null 인 토론 검색
2	본인이 생성한 토론인지 확인 (자기 자신 참여 차단)
3	opponent_id 설정 + status를 `arguing` 으로 변경

RESPONSE `200`

업데이트된 토론 객체 반환

ERROR

`400`	본인이 생성한 토론에 참여 시도
`404`	초대 코드 없음 또는 이미 진행 중인 토론

POST

/api/debate/:id/view

PUBLIC

조회수 증가

설명

해당 토론의 조회수를 1 증가시킵니다.

PATH PARAMS

이름	타입	설명
`id`	UUID	토론 ID

RESPONSE 200

```
{ "message": "조회수가 증가되었습니다." }
```

POST

/api/debate/:id/like

AUTH

좋아요 토글

설명

해당 토론에 좋아요를 추가하거나 이미 좋아요한 경우 취소합니다 (토글).

RESPONSE 200

```
{
  "liked": true,
  "like_count": 15
}
```

POST

/api/debate/batch/likes

AUTH

좋아요 일괄 조회

설명

여러 토론에 대한 현재 사용자의 좋아요 상태를 일괄 조회합니다.

REQUEST BODY

```
{
  "debateIds": ["uuid1", "uuid2", "uuid3"]
}
```

RESPONSE 200

```
{
  "uuid1": true,
  "uuid2": false,
  "uuid3": true
}
```

GET

/api/debate/:id/like/me

AUTH

내 좋아요 상태 조회

설명

해당 토론에 대한 현재 사용자의 좋아요 상태를 조회합니다.

RESPONSE 200

```
{
  "liked": true
}
```

DELETE

/api/debate/:id

AUTH

토론 삭제

설명

토론 생성자만 토론을 삭제할 수 있습니다.

RESPONSE 200

```
{ "message": "토론이 삭제되었습니다." }
```

ERROR

403	토론 생성자가 아닌 경우
404	토론을 찾을 수 없음

6. 주장 API (/api/argument) — 3개

GET

`/api/argument/:debateId`

OPTIONALAUTH

주장 목록 조회

설명

해당 토론의 주장 목록을 조회합니다. 인증 없이도 접근 가능하며, 인증 시 추가 정보 제공.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "debate_id": "uuid",
    "user_id": "uuid",
    "side": "A",
    "content": "주장 내용",
    "created_at": "ISO8601",
    "user": { "nickname": "닉네임" }
  }
]
```

POST

/api/argument/:debateId

AUTH

CONTENTFILTER

주장 제출

REQUEST BODY

```
{
  "content": "주장 내용 (50~2,000자)",
  "side": "A",
  "category": "daily",
  "topic": "토론 주제"
}
```

처리 파이프라인

```
사용자 입력
    │
    ▼
┌──────────────────────────┐
│ 1. contentFilter 미들웨어  │ ← 비속어/AI유해성/주제적합성
└──────────────┬───────────┘
               │ pass
               ▼
┌──────────────────────────┐
│ 2. preprocessArgument()   │ ← 인젝션 방어 + 정규화 + 유사도
│   - 2,000자 초과 → 자름    │
│   - 인젝션 패턴 → [필터링됨] │
│   - HTML/코드블록 제거     │
│   - 공백/개행 정리         │
│   - 50자 미만 → 거부       │
│   - 상대측 유사도 85%↑ → 거부 │
└──────────────┬───────────┘
               │ valid
               ▼
┌──────────────────────────┐
│ 3. DB 저장 (정제된 텍스트)  │
└──────────────────────────┘
```

RESPONSE 201

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "side": "A",
  "content": "전처리 완료된 주장 텍스트",
```

```
  "created_at": "ISO8601",
  "warnings": ["프롬프트 인젝션 시도 2건 필터링됨"]
}
```

ERROR

400	콘텐츠 필터 차단 (Stage 1/2/3)
400	"주장은 50자 이상이어야 합니다"
400	"상대측과 동일한 주장은 제출할 수 없습니다"

POST

/api/argument/:debateId/solo

AUTH

Solo AI 반론 생성

설명

Solo 모드에서 AI가 B측 반대 주장을 자동 생성합니다.

비즈니스 로직

1	토론 존재 여부 + status=arguing 확인
2	요청자가 토론 생성자(creator)인지 확인
3	A측 주장 존재 + B측 주장 미존재 확인
4	AI가 반대 주장(B측)을 생성하여 저장

RESPONSE 201

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": null,
  "side": "B",
  "content": "AI가 생성한 반대 주장 텍스트",
  "ai_generated": true,
  "created_at": "ISO8601"
}
```

ERROR

400	토론이 주장 제출 단계가 아님 / A측 주장 없음
403	토론 생성자가 아닌 경우
409	이미 B측 주장이 존재

7. 채팅 API (/api/chat) — 7개

GET

/api/chat/:debateId/messages

AUTH

채팅 메시지 조회

설명

해당 토론의 채팅 메시지 목록을 조회합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "debate_id": "uuid",
    "user_id": "uuid",
    "content": "메시지 내용",
    "side": "A",
    "created_at": "ISO8601",
    "user": { "nickname": "닉네임", "avatar_url": "..." }
  }
]
```

POST

/api/chat/:debateId/messages

AUTH

메시지 전송

REQUEST BODY

```
{
  "content": "메시지 내용",
  "side": "A"
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "content": "메시지 내용",
  "side": "A",
  "created_at": "ISO8601"
}
```

POST

/api/chat/:debateId/start

AUTH

채팅 시작

설명

토론 채팅 세션을 시작합니다.

RESPONSE 200

```
{ "message": "채팅이 시작되었습니다." }
```

POST

/api/chat/:debateId/end

AUTH

채팅 종료

설명

토론 채팅 세션을 종료합니다.

RESPONSE 200

```
{ "message": "채팅이 종료되었습니다." }
```

POST

/api/chat/:debateId/report

AUTH

사용자 신고

설명

채팅 중 부적절한 사용자를 신고합니다.

REQUEST BODY

```
{
  "reported_user_id": "uuid",
  "reason": "신고 사유"
}
```

RESPONSE 200

```
{ "message": "신고가 접수되었습니다." }
```

POST

/api/chat/:debateId/citizen-vote

AUTH

시민 투표

설명

채팅 토론에서 시민 투표를 진행합니다.

REQUEST BODY

```
{
  "voted_side": "A"
}
```

RESPONSE 200

```
{
  "voted_side": "A",
  "message": "투표가 완료되었습니다."
}
```

GET

/api/chat/:debateId/citizen-vote

AUTH

시민 투표 집계

설명

해당 채팅 토론의 시민 투표 결과를 집계하여 반환합니다.

RESPONSE 200

```
{
  "A": 12,
  "B": 8,
  "total": 20
}
```

8. 판결 API (/api/judgment) — 8개

GET

`/api/judgment/feed`

PUBLIC

판결 피드

QUERY PARAMETERS

이름	타입	기본값	설명
page	Number	1	페이지 번호
limit	Number	10	페이지당 항목 수

RESPONSE 200

```
{
  "data": [
    {
      "id": "uuid",
      "winner_side": "A",
      "summary": "...",
      "ai_score_a": 78, "ai_score_b": 65,
      "final_score_a": 78, "final_score_b": 65,
      "is_citizen_applied": true,
      "debate": {
        "topic": "짜장면 vs 짬뽕",
        "category": "일상",
        "creator": { "nickname": "닉네임" }
      }
    }
  ],
  "page": 1,
  "limit": 10,
  "hasNext": true
}
```

GET

/api/judgment/hall

PUBLIC

명예의 전당

설명

높은 점수/관심도를 받은 우수 판결 목록을 조회합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "debate": { "topic": "...", "category": "..." },
    "winner_side": "A",
    "final_score_a": 92,
    "final_score_b": 71,
    "summary": "..."
  }
]
```

GET

/api/judgment/daily

PUBLIC

일일 판결 조회

설명

오늘의 일일 토론에 대한 판결을 조회합니다.

RESPONSE 200

판결 객체 반환 (일일 토론이 없으면 null)

GET

/api/judgment/:debateId/og

PUBLIC

OG 메타 데이터

설명

판결 공유를 위한 OG 메타 데이터를 반환합니다.

RESPONSE 200

```
{
  "title": "모라고라 판결 - 짜장면 vs 짬뽕",
  "description": "A측 승리 (78 vs 65)",
  "image": "https://..."
}
```

GET

/api/judgment/:debateId

PUBLIC

판결 상세 조회

RESPONSE 200

verdict + ai_judgments 전체 객체 반환

POST

/api/judgment/:debateId

AUTH

AI 판정 요청

사전 검증

1	토론 상태가 `arguing` 인지 확인
2	이미 판결(verdict)이 존재하는지 확인
3	양측 주장(A, B)이 모두 존재하는지 확인

비즈니스 로직

1	status → `judging` 원자적 전환
2	3개 AI 병렬 호출 (Promise.allSettled) — 실패 시 Grok 대체
3	개별 AI 판결을 ai_judgments에 저장
4	종합 판결 계산 + status → `voting`

RESPONSE `200`

```
{
  "id": "verdict-uuid",
  "debate_id": "uuid",
  "winner_side": "A",
  "summary": "A측의 구체적인 사례 제시가 결정적이었습니다...",
  "ai_score_a": 78, "ai_score_b": 65,
  "ai_judgments": [
    {
      "ai_model": "gpt-4o",
      "winner_side": "A",
      "score_a": 80, "score_b": 63,
      "confidence": 0.78
    }
  ],
  "vote_deadline": "2026-04-03T00:00:00Z"
}
```

ERROR

`400`	토론이 주장 제출 단계가 아님 / 양측 주장 미제출
`409`	이미 판결이 존재

500	모든 AI 모델 실패 (Grok 포함)

POST

/api/judgment/:debateId/retry/:model

AUTH

모델별 재판정

설명

특정 AI 모델의 판결을 다시 요청합니다. 기존 해당 모델의 판결을 대체합니다.

PATH PARAMS

이름	타입	설명
debateId	UUID	토론 ID
model	String	gpt-4o / gemini-2.5-flash / claude-sonnet / grok-3-mini

RESPONSE 200

재판정된 AI 판결 객체 반환

POST

/api/judgment/:debateId/rate

AUTH

판결 별점 평가

설명

판결 결과에 대한 별점 평가를 제출합니다.

REQUEST BODY

```
{
  "rating": 4
}
```

RESPONSE 200

```
{
  "rating": 4,
  "message": "평가가 등록되었습니다."
}
```

9. 투표 API (/api/vote) — 6개

POST

`/api/vote/finalize`

AUTH

만료 투표 마감

비즈니스 로직 (FINALIZEVERDICT)

1	status=voting AND vote_deadline < now 인 토론 검색
2	투표 30건 이상 → 시민 점수 반영 (AI 75% + 시민 25%)
3	투표 30건 미만 → AI 점수만 사용
4	최종 승자 결정 + status → `completed`
5	양측 프로필 전적 자동 업데이트

RESPONSE `200`

```
{
  "finalized": 3,
  "results": [
    {
      "debateId": "uuid",
      "finalA": 76, "finalB": 68,
      "finalWinner": "A",
      "citizenApplied": true,
      "voterCount": 45
    }
  ]
}
```

POST

/api/vote/batch/my

AUTH

내 투표 일괄 조회

설명

여러 토론에 대한 현재 사용자의 투표 상태를 일괄 조회합니다.

REQUEST BODY

```
{
  "debateIds": ["uuid1", "uuid2", "uuid3"]
}
```

RESPONSE 200

```
{
  "uuid1": { "voted_side": "A" },
  "uuid2": null,
  "uuid3": { "voted_side": "B" }
}
```

GET

/api/vote/:debateId/my

AUTH

내 투표 조회

설명

해당 토론에 대한 현재 사용자의 투표를 조회합니다.

RESPONSE 200

```
{
  "voted_side": "A",
  "created_at": "ISO8601"
}
```

GET

/api/vote/:debateId

PUBLIC

투표 집계 조회

RESPONSE 200

```
{
  "A": 15,
  "B": 12,
  "total": 27,
  "status": "voting",
  "vote_deadline": "2026-04-03T00:00:00Z"
}
```

POST

/api/vote/:debateId

AUTH

투표

REQUEST BODY

```
{
  "voted_side": "A"
}
```

RESPONSE 200

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "voted_side": "A",
  "created_at": "ISO8601"
}
```

Upsert: 이미 투표한 경우 투표를 변경할 수 있습니다 (마감 전까지).

ERROR

400	"voted_side는 'A' 또는 'B'만 허용됩니다"
400	"투표 기간이 아닙니다" / "투표가 마감되었습니다"

DELETE

/api/vote/:debateId

AUTH

투표 취소

설명

현재 사용자의 투표를 취소합니다. 투표 기간(status=voting)에만 가능합니다.

RESPONSE 200

```
{ "message": "투표가 취소되었습니다." }
```

ERROR

400	"투표 기간이 아닙니다."

10. 댓글 API (/api/comment) — 3개

GET

`/api/comment/:debateId`

OPTIONALAUTH

댓글 조회

설명

해당 토론의 댓글 목록을 조회합니다. 인증 없이도 접근 가능합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "debate_id": "uuid",
    "user_id": "uuid",
    "content": "댓글 내용",
    "created_at": "ISO8601",
    "user": { "nickname": "닉네임", "avatar_url": "..." }
  }
]
```

POST

/api/comment/:debateId

AUTH

COMMENTFILTER

댓글 작성

REQUEST BODY

```
{
  "content": "댓글 내용"
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "debate_id": "uuid",
  "user_id": "uuid",
  "content": "댓글 내용",
  "created_at": "ISO8601"
}
```

ERROR

400	댓글 필터에 의해 차단

DELETE

/api/comment/:commentId

AUTH

댓글 삭제

설명

본인이 작성한 댓글을 삭제합니다.

RESPONSE 200

```
{ "message": "댓글이 삭제되었습니다." }
```

ERROR

403	본인 댓글이 아닌 경우
404	댓글을 찾을 수 없음

11. 프로필 API (/api/profile) — 12개

GET

/api/profile/ranking

PUBLIC

랭킹 조회

RESPONSE 200

```
[
  {
    "id": "uuid",
    "nickname": "1위유저",
    "avatar_url": "...",
    "wins": 15, "losses": 3, "draws": 2,
    "total_score": 1200,
    "tier": "변호사"
  }
]
```

total_score 기준 내림차순 정렬, 상위 50명

GET

/api/profile/me

AUTH

내 프로필 조회

RESPONSE 200

```
{
  "id": "uuid",
  "nickname": "닉네임",
  "gender": "male",
  "age": 25,
  "avatar_url": "...",
  "wins": 5, "losses": 3, "draws": 1,
  "total_score": 420,
  "tier": "배심원"
}
```

PATCH

/api/profile/me

AUTH

VALIDATE

프로필 수정

REQUEST BODY

```
{
  "nickname": "새닉네임",
  "gender": "male",
  "age": 26,
  "avatar_url": "..."
}
```

RESPONSE 200

수정된 프로필 객체 반환

PATCH

/api/profile/me/onboarding

AUTH

온보딩 완료

설명

사용자의 온보딩 완료 상태를 업데이트합니다.

RESPONSE 200

```
{ "message": "온보딩이 완료되었습니다." }
```

GET

/api/profile/me/role

AUTH

내 역할 조회

RESPONSE 200

```
{
  "role": "user",
  "tier": "배심원"
}
```

GET

/api/profile/me/verdicts

AUTH

내 판결 조회

RESPONSE 200

```
[
  {
    "id": "debate-uuid",
    "topic": "짜장면 vs 짬뽕",
    "status": "completed",
    "verdicts": [
      {
        "winner_side": "A",
        "ai_score_a": 78, "ai_score_b": 65,
        "final_score_a": 76, "final_score_b": 68
      }
    ]
  }
]
```

GET

/api/profile/me/debates

AUTH

내 토론 조회

설명

현재 사용자가 참여한 모든 토론 목록을 조회합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "topic": "...",
    "status": "completed",
    "mode": "duo",
    "created_at": "ISO8601"
  }
]
```

GET

/api/profile/me/analysis

AUTH

내 분석 조회

설명

사용자의 토론 성향 분석 데이터를 조회합니다.

RESPONSE 200

```
{
  "total_debates": 20,
  "win_rate": 0.65,
  "avg_score": 75.3,
  "strong_categories": ["사회", "기술"],
  "rubric_avg": {
    "logic": 16.2,
    "evidence": 14.8,
    "persuasion": 15.1,
    "consistency": 15.5,
    "expression": 14.9
  }
}
```

DELETE

/api/profile/me/verdicts/:debateId

AUTH

내 토론 삭제

설명

내 판결 기록에서 특정 토론을 삭제합니다.

RESPONSE 200

```
{ "message": "삭제되었습니다." }
```

DELETE

/api/profile/me

AUTH

계정 삭제

설명

현재 사용자의 계정을 영구 삭제합니다. 관련 데이터(토론, 주장, 투표 등)가 함께 정리됩니다.

RESPONSE 200

```
{ "message": "계정이 삭제되었습니다." }
```

주의: 이 작업은 되돌릴 수 없습니다.

GET

/api/profile/me/xp-logs

AUTH

내 XP 로그

RESPONSE 200

```
[
  {
    "id": "uuid",
    "user_id": "uuid",
    "reason": "victory",
    "xp_change": 30,
    "created_at": "ISO8601"
  }
]
```

reason 값: victory(+30), defeat(+5), draw(+15), daily_debate(+10), vote(+3/-3), bonus, penalty

GET

/api/profile/:userId

PUBLIC

공개 프로필 조회

RESPONSE 200

```
{
  "id": "uuid",
  "nickname": "닉네임",
  "avatar_url": "string | null",
  "wins": 5, "losses": 3, "draws": 1,
  "total_score": 420,
  "tier": "배심원"
}
```

12. 알림 API (/api/notification) — 6개

GET

/api/notification/

AUTH

알림 목록

설명

현재 사용자의 알림 목록을 조회합니다.

RESPONSE 200

```
[
  {
    "id": "uuid",
    "user_id": "uuid",
    "type": "judgment_complete",
    "title": "판결이 완료되었습니다",
    "message": "짜장면 vs 짬뽕 토론의 판결이 나왔습니다.",
    "reference_id": "debate-uuid",
    "is_read": false,
    "created_at": "ISO8601"
  }
]
```

GET

/api/notification/unread-count

AUTH

읽지않음 수

RESPONSE 200

```
{
  "count": 5
}
```

PATCH

/api/notification/:id/read

AUTH

읽음 처리

설명

특정 알림을 읽음 처리합니다.

RESPONSE 200

```
{ "message": "읽음 처리되었습니다." }
```

PATCH

/api/notification/read-all

AUTH

전체 읽음

설명

모든 알림을 읽음 처리합니다.

RESPONSE 200

```
{ "message": "모든 알림이 읽음 처리되었습니다." }
```

DELETE

/api/notification/:id

AUTH

알림 삭제

설명

특정 알림을 삭제합니다.

RESPONSE 200

```
{ "message": "알림이 삭제되었습니다." }
```

DELETE

/api/notification/

AUTH

전체 삭제

설명

모든 알림을 삭제합니다.

RESPONSE 200

```
{ "message": "모든 알림이 삭제되었습니다." }
```

13. 콘텐츠 필터 API (/api/content) — 2개

POST

`/api/content/check`

AUTH

콘텐츠 검사

REQUEST BODY

```
{
  "content": "검사할 텍스트",
  "category": "사회",
  "debateId": "uuid"
}
```

3단계 필터 기준

카테고리	Stage 1 (사전)	Stage 2 (AI 유해성)	Stage 3 (주제 적합성)
일상, 연애	O	-	-
직장, 교육	O	O	-
사회, 정치	O	O	O

RESPONSE 200

```
// 통과
{ "pass": true, "stage": 1 }

// 차단
{ "pass": false, "stage": 2, "reason": "혐오 발언 감지" }
```

GET

/api/content/logs

AUTH

필터 로그 조회

RESPONSE 200

```
[
  {
    "id": "uuid",
    "content_type": "argument",
    "filter_stage": 1,
    "result": "block",
    "reason": "금지어 포함",
    "created_at": "ISO8601"
  }
]
```

14. 피드백 API (/api/feedback) — 2개

POST

/api/feedback/

AUTH

피드백 제출

설명

서비스 만족도 피드백을 제출합니다. 기존 피드백이 있으면 수정(upsert), 없으면 새로 생성합니다.

REQUEST BODY

```
{
  "satisfaction": 4,
  "ai_accuracy": 4,
  "ui_ease": 3,
  "fairness": 4,
  "recommend": 5,
  "best_feature": "AI 판결이 재밌어요",
  "improvement": "로딩이 느려요",
  "additional": "추가 의견"
}
```

RESPONSE 201

```
{
  "id": "uuid",
  "user_id": "uuid",
  "satisfaction": 4,
  "ai_accuracy": 4,
  "ui_ease": 3,
  "fairness": 4,
  "recommend": 5,
  "created_at": "ISO8601"
}
```

ERROR

400	"각 항목은 1~5 사이 정수 값이어야 합니다."

GET

/api/feedback/me

AUTH

내 피드백 조회

RESPONSE 200

```
[
  {
    "id": "uuid",
    "satisfaction": 4,
    "ai_accuracy": 4,
    "ui_ease": 3,
    "fairness": 4,
    "recommend": 5,
    "best_feature": "...",
    "improvement": "...",
    "created_at": "ISO8601"
  }
]
```

15. 분석 API (/api/analytics) — 2개

POST

`/api/analytics/page-view`

OPTIONALAUTH

페이지뷰 기록

설명

페이지 조회 이벤트를 기록합니다. 비인증 사용자도 기록 가능합니다.

REQUEST BODY

```
{
  "path": "/debate/uuid",
  "referrer": "https://..."
}
```

RESPONSE 200

```
{ "message": "기록되었습니다." }
```

POST

/api/analytics/event

OPTIONALAUTH

이벤트 기록

설명

커스텀 분석 이벤트를 기록합니다.

REQUEST BODY

```
{
  "event_name": "debate_created",
  "event_data": {
    "category": "일상",
    "mode": "duo"
  }
}
```

RESPONSE 200

```
{ "message": "기록되었습니다." }
```

16. 관리자 API (/api/admin) — 4개 ▼

권한: 모든 관리자 API는 AUTH + ADMIN 권한이 필요합니다.

GET

`/api/admin/stats`

AUTH

ADMIN

대시보드 통계

설명

관리자 대시보드에 표시할 전체 통계 데이터를 조회합니다.

RESPONSE 200

```
{
  "total_users": 1250,
  "total_debates": 830,
  "total_verdicts": 780,
  "total_votes": 5600,
  "active_debates": 45,
  "today_signups": 12
}
```

GET

/api/admin/ai

AUTH

ADMIN

AI 통계

설명

AI 모델별 판결 통계를 조회합니다.

RESPONSE 200

```
{
  "models": {
    "gpt-4o": { "count": 750, "avg_confidence": 0.76, "fail_rate": 0.02 },
    "gemini-2.5-flash": { "count": 740, "avg_confidence": 0.74, "fail_rate": 0.03
},
    "claude-sonnet": { "count": 735, "avg_confidence": 0.75, "fail_rate": 0.04 },
    "grok-3-mini": { "count": 55, "avg_confidence": 0.72, "fail_rate": 0.01 }
  }
}
```

GET

/api/admin/trends

AUTH

ADMIN

일별 트렌드

설명

일별 사용자 가입, 토론 생성, 판결 완료 등의 트렌드 데이터를 조회합니다.

RESPONSE 200

```
[
  {
    "date": "2026-04-01",
    "signups": 15,
    "debates_created": 28,
    "verdicts_completed": 22,
    "votes_cast": 180
  }
]
```

GET

/api/admin/analytics

AUTH

ADMIN

분석 데이터

설명

페이지뷰, 이벤트 등 분석 데이터를 관리자용으로 집계하여 반환합니다.

RESPONSE 200

```
{
  "page_views": {
    "total": 25000,
    "today": 450,
    "top_pages": [
      { "path": "/", "count": 5000 },
      { "path": "/debate", "count": 3200 }
    ]
  },
  "events": {
    "debate_created": 830,
    "judgment_requested": 780,
    "vote_cast": 5600
  }
}
```

17. AI API (/api/ai) — 3개

POST

`/api/ai/generate-sides`

PUBLIC

A/B측 자동 생성

설명

토론 주제를 입력하면 AI가 A측과 B측의 입장을 자동으로 생성합니다.

REQUEST BODY

```
{
  "topic": "짜장면 vs 짬뽕, 어느 쪽이 더 나은 선택인가?"
}
```

RESPONSE 200

```
{
  "sideA": "짜장면을 지지하는 입장...",
  "sideB": "짬뽕을 지지하는 입장..."
}
```

POST

/api/ai/socratic-feedback

AUTH

소크라틱 피드백

설명

사용자의 주장에 대해 AI가 소크라테스 대화법 기반의 질문/피드백을 생성합니다.

REQUEST BODY

```
{
  "topic": "토론 주제",
  "content": "사용자의 주장 내용",
  "side": "A"
}
```

RESPONSE 200

```
{
  "feedback": "그 주장의 근거가 되는 구체적인 사례를 들어볼 수 있을까요?",
  "suggestions": [
    "논리 구조를 강화해 보세요",
    "구체적인 데이터를 추가하면 설득력이 높아집니다"
  ]
}
```

POST

/api/ai/rubric-score

AUTH

실시간 루브릭 점수

설명

작성 중인 주장에 대해 AI가 실시간으로 5항목 루브릭 점수를 예측합니다.

REQUEST BODY

```
{
  "topic": "토론 주제",
  "content": "작성 중인 주장 내용",
  "side": "A"
}
```

RESPONSE 200

```
{
  "total": 72,
  "detail": {
    "logic": 15,
    "evidence": 14,
    "persuasion": 15,
    "consistency": 14,
    "expression": 14
  },
  "tips": "근거를 뒷받침하는 구체적 사례를 추가하면 점수가 올라갈 수 있습니다."
}
```

18. OG 메타 API (/api/og) — 1개

GET

`/api/og/invite/:inviteCode`

PUBLIC

초대 OG 메타태그 HTML

설명

초대 코드로 토론 정보를 조회하여 동적 OG 메타태그가 포함된 HTML을 반환합니다. 카카오톡/Discord/슬랙 등에서 링크 미리보기(OG 카드)에 토론 주제가 표시됩니다. 브라우저 접속 시 `<meta http-equiv="refresh">`로 클라이언트 초대 페이지로 자동 리다이렉트됩니다.

PATH PARAMETER

이름	타입	설명
inviteCode	String	초대 코드 (8자리)

RESPONSE `200` (TEXT/HTML)

```
<!DOCTYPE html>
<html>
<head>
  <meta property="og:title" content="모라고라 - {토론 주제}" />
  <meta property="og:description" content="{카테고리} | {목적} 논쟁에 초대합니다"
/>
  <meta property="og:image" content="https://team-moragora-client.vercel.app/og-c
ard.png" />
  <meta http-equiv="refresh" content="0;url=https://team-moragora-client.vercel.a
pp/invite/{inviteCode}" />
</head>
</html>
```

19. 크론 API (/api/cron) — 3개

권한: 모든 크론 API는 VERIFYCRONSECRET 미들웨어로 보호됩니다. 요청 헤더에 크론 비밀키가 필요합니다.

POST

/api/cron/daily-debate

CRON

일일 토론 생성

설명

매일 자동으로 일일 토론을 생성합니다. 크론 스케줄러에 의해 호출됩니다.

RESPONSE 201

```
{
  "id": "uuid",
  "topic": "오늘의 토론 주제",
  "category": "daily",
  "status": "waiting",
  "created_at": "ISO8601"
}
```

DELETE

/api/cron/daily-debate/:id

CRON

일일 토론 삭제

설명

특정 일일 토론을 삭제합니다.

RESPONSE 200

```
{ "message": "일일 토론이 삭제되었습니다." }
```

POST

/api/cron/cleanup-stale

CRON

오래된 토론 정리

설명

일정 기간 이상 방치된 대기(waiting) 상태의 토론을 자동으로 정리합니다.

RESPONSE 200

```
{
  "cleaned": 5,
  "message": "5개의 오래된 토론이 정리되었습니다."
}
```

20. AI 판결 시스템 ▼

AI 판사 캐릭터

AI	캐릭터	모델	성격	말투
Judge G	분석적 판사	GPT-4o	균형잡힌, 분석적, 정중한	격식체 (~입니다, ~하겠습니다)
Judge M	통찰적 판사	Gemini 2.5 Flash	다각적, 창의적, 통찰력	친근체 (~거든요, ~잖아요)
Judge C	배려하는 판사	Claude Sonnet 4	신중한, 공정한, 배려하는	부드러운 존댓말 (~네요, ~것 같아요)
Judge X	대체 판사	Grok 3 Mini	Judge G 스타일	격식체 (폴백용)

판결 프롬프트 구조 (System / User 분리)

```
┌──────────────────────────────────────────┐
│              System Prompt               │
│  ┌──────────────────────────────────┐    │
│  │ 캐릭터 설정 (Judge G/M/C 고유 성격)  │   │
│  │ + 판결문 톤 예시                    │   │
│  │ + 판결 원칙 (6개)                   │   │
│  │ + 5항목 루브릭 (각 20점)            │   │
│  │ + confidence 기준표 (0.50~1.00)     │   │
│  │ + 금지 사항 (6개)                   │   │
│  │ + JSON 응답 스키마                  │   │
│  └──────────────────────────────────┘    │
├──────────────────────────────────────────┤
│              User Prompt                 │
│  ┌──────────────────────────────────┐    │
│  │ 논쟁 주제 (topic)                   │   │
│  │ + 판결 목적 (purpose)               │   │
│  │ + 분석 렌즈 (lens + 가중치 설명)     │   │
│  │ + A측 주장 원문                     │   │
│  │ + B측 주장 원문                     │   │
│  └──────────────────────────────────┘    │
└──────────────────────────────────────────┘
```

판결 목적 (purpose)

값	설명	판결 방식
`compete`	승부 판별	논리적 우열 → 승/패/무 판정 + 점수
`consensus`	합의 도출	양측 공통 접점과 합리적 결론 도출
`analysis`	분석 요청	양측 논리를 객관적으로 정리 비교

분석 렌즈 (lens) — 가중치 조정

값	설명	가중 항목
`general`	종합 관점	5개 항목 균등 (각 20점)
`logic`	논리/팩트	logic 항목에 가중치
`emotion`	관계/감정	persuasion 항목에 가중치
`practical`	실용/비용	evidence 항목에 가중치
`ethics`	윤리/가치	consistency 항목에 가중치

confidence 기준표

구간	의미	기준
0.90 ~ 1.00	압도적 차이	한쪽 구체적 근거 다수 vs 상대측 근거 전무
0.80 ~ 0.89	명확한 차이	양측 근거 있으나 한쪽 논리 구조가 확실히 우수
0.70 ~ 0.79	분명한 차이	양측 유사 수준이나 핵심 논점에서 한쪽 우위
0.55 ~ 0.69	근소한 차이	양측 모두 우수하나 미세한 차이로 판정
0.50 ~ 0.54	사실상 동점	draw 판정 시에만 사용

AI API 설정

항목	GPT-4o	Gemini 2.5 Flash	Claude Sonnet 4	Grok 3 Mini
temperature	0.3	0.3	-	0.3
max_tokens	-	-	1,024	-
timeout	30초	30초	30초	30초
JSON 강제	response_format	responseMimeType	regex 파싱	response_format
prompt 분리	system + user	systemInstruction + contents	system + messages	system + user

Solo 모드 — AI 반대 주장 생성

항목	설정
사용 모델	GPT-4o-mini (단독)
생성 길이	200~500자
temperature	0.7 (다양한 반대 의견)
저장	arguments 테이블, user_id: null, ai_generated: true

엔드포인트	`POST /api/argument/:debateId/solo`
판결 시 공개	AI 생성 사실 비공개 (공정성)

21. 에러 코드 정리

코드	의미	발생 상황
200	OK	정상 조회, 업데이트 성공
201	Created	토론/주장/투표/댓글/피드백 생성 성공
400	Bad Request	콘텐츠 필터 차단, 유효성 검증 실패, 토론 상태 불일치, 투표 마감 초과
401	Unauthorized	JWT 토큰 없음 또는 만료, 크론 비밀키 불일치
403	Forbidden	권한 없음 (관리자 전용, 본인 리소스 아님, Solo 모드 생성자 아님)
404	Not Found	토론/사용자/초대코드/댓글/알림 없음
409	Conflict	중복 판결 요청, Solo 모드 B측 주장 중복
500	Internal Error	DB 오류, 모든 AI 모델 실패 (status → arguing 롤백)

공통 에러 응답 형식

```
{
  "error": "에러 메시지 (한국어)"
}
```